

NEON DRIFT

게임 기획서

1인 개발 포트폴리오

Unreal Engine 5 · C++ · 3일 개발

액션 리소스 디펜스

목차

1. 게임 개요

항목	내용
타이틀	NEON DRIFT
장르	액션 리소스 디펜스
엔진	Unreal Engine 5 (C++)
개발 인원	1인
개발 기간	3일
플랫폼	PC (Windows)

컨셉 한 줄

호버크래프트를 조종해 기지를 방어하는 네온 아케이드 디펜스

게임 설명

NEON DRIFT는 플레이어가 호버크래프트를 직접 조종하며 웨이브로 몰려오는 몬스터로부터 기지를 지키는 1인칭 아케이드 디펜스 게임이다. 일반적인 타워 디펜스와 달리, 플레이어는 맵을 자유롭게 비행하면서 자원 블록을 채취하고, 수동 터렛에 직접 탑승해 방어하거나 자동 터렛을 업그레이드하는 등 능동적으로 전선을 구성한다. 웨이브 사이 상점에서 업그레이드를 구매하며 5웨이브를 버텨내는 것이 목표다.

핵심 루프

비행·자원 채취 → 웨이브 방어 → 상점 업그레이드 → 반복

2. 핵심 메카닉

2-1. 호버크래프트 비행 조작

플레이어는 중력 없이 6방향 자유 비행이 가능한 호버크래프트를 조종한다.

- 이동: WASD로 수평 이동, Space로 상승 / C로 하강
- 물리: 중력 비활성화, 전 축 균일 드래그 적용 – 입력을 놓으면 서서히 감속
- 자동 고도 유지: Z축 속도가 자연 감쇠하여 일정 고도에서 부유하는 느낌 구현
- 카메라: 선체와 독립된 Pitch/Yaw 회전 (마우스), 속도에 따라 FOV 동적 변화 (70°~110°)
- 조준: 카메라 방향 기준 선 추적(Line Trace)으로 사격 – 크로스헤어와 실제 탄착점 일치

2-2. 자원 채취 시스템

맵에 배치된 자원 블록을 파괴해 자원 파편(Shard)을 수집하고, 수집한 자원으로 상점에서 업그레이드를 구매한다.

자원 블록 3종

종류	공격력 요구치	특징
Gold	낮음	기본 자원, 초반 수급원
Iron	중간	터렛 관련 업그레이드 재료
Diamond	높음	고급 업그레이드 전용

- 블록 파괴 시 Shard 여러 개가 분산 생성됨
- Shard는 플레이어 근접 시 자석 호밍 작동 – 일정 거리 안에 들어오면 가속하며 자동 흡수
- 마그네틱 반경과 흡수 속도는 업그레이드로 강화 가능

2-3. 터렛 시스템

기지 방어에 핵심. 수동 터렛(직접 조작)과 자동 터렛(AI 자동 사격) 두 종류가 존재한다.

수동 터렛 (Manual Turret)

- 탑승: 터렛 근처에서 E 키 → 카메라가 터렛 후방으로 블렌딩 전환 (0.25초)
- 조준: 마우스로 포신 방향 제어, 회전 속도 제한으로 급격한 꺾임 방지
- 사격: LMB — 카메라 위치 기준, 포신 전방 방향으로 선 추적 발사
- HUD 이중 크로스헤어: 화면 중앙 흰색(카메라/목표 방향) + 노란 점(실제 포신 방향) — 두 크로스헤어가 일치할 때 정확히 명중
- 하차: E 키 재입력 → 선체 복귀, 카메라 블렌딩 복원

- 탑승 중에는 선체가 완전 정지 (속도 0 고정)

자동 터렛 (Auto Turret)

- 가장 가까운 몬스터를 자동 탐지하여 선 추적으로 사격
- 웨이브 중 SetEnabled 제어로 활성화/비활성 전환 가능
- 상점에서 공격력·사거리·발사 속도 업그레이드 가능

2-4. 웨이브 방어

총 5웨이브로 구성되며, 모든 웨이브를 버티면 승리한다.

웨이브 구성

웨이브	몬스터 수	몬스터 HP	스폰 간격
1	5	1	2.0초
2	10	1	1.5초
3	15	2	1.0초
4	20	2	1.0초
5	30	3	0.8초

- 몬스터는 기지를 향해 이동 (속도 350uu/s), 사거리 2000uu 진입 시 원거리 공격 시작
- 공중 몬스터 포함: 기지 상공 500uu 고도로 접근하는 별도 패턴

2-5. 업그레이드 상점

매 웨이브 클리어 후 상점 페이지 진입. 수집한 자원으로 업그레이드를 구매한다.

- 조작: ↑↓ 항목 탐색, Enter 구매, Tab 다음 웨이브 진행
- 구매 가능 여부는 자원량 실시간 체크, 부족 시 비활성 표시
- 업그레이드 상태는 게임 인스턴스에 영구 보존 (웨이브 간 유지)

업그레이드 목록

ID	이름	효과	최대 레벨
Ship_Magnet	마그넷 범위	자석 수집 반경 확대	4
Ship_Attack	선체 공격력	공격력 증가 (블록 채굴 게이트 해제)	4

ID	이름	효과	최대 레벨
Ship_Speed	선체 속도	최대 이동속도 증가	4
Ship_HP	선체 HP	최대 HP 증가	4
Turret_Rotate	터렛 회전속도	포신 회전 속도 증가	4
Turret_Attack	터렛 공격력	수동 터렛 데미지 증가	4
Turret_FireRate	터렛 발사속도	수동 터렛 연사속도 증가	4
AutoTurret_Count	자동 터렛 +1	활성 자동 터렛 수 증가	4

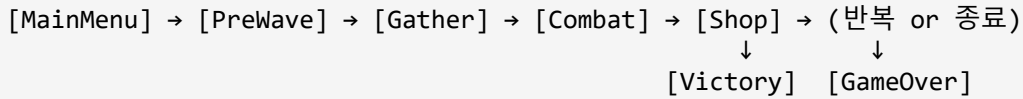
2-6. 기지 HP 시스템

- 몬스터의 공격이 기지에 도달하면 HP 감소
- 피격 시 기지 머티리얼이 빨간색 플래시로 시각 피드백
- HP 25% 이하 시 빨간색 경고 상태로 전환
- HP 0 → 게임오버

3. 시스템 설계

3-1. 게임 루프 상태 머신

게임 진행은 GameMode가 관리하는 7단계 상태 머신으로 구성된다.



상태	조건	동작
MainMenu	게임 시작	타이틀 화면, Start Game/Controls/Quit 메뉴
PreWave	StartGame 호출	맵 초기화, 자원 블록 배치, 스폰 포인트 설정
Gather	PreWave 완료	몬스터 없음, 자원 채취만 가능 – F키로 조기 진입 가능
Combat	Gather 종료	웨이브 정의에 따라 몬스터 스폰 펌프 가동, 기지 방어
Shop	웨이브 전원 처치	상점 UI 활성화, 업그레이드 구매 후 Tab으로 다음 웨이브
Victory	5웨이브 클리어	승리 오버레이 표시
GameOver	기지 HP 0	게임오버 오버레이, R키로 재시작

3-2. 자원 필드 스폰 구조

- GameMode가 PreWave 진입 시 맵에 자원 블록을 일괄 배치
- 블록은 기지 Z축 기준 +700uu 상공에 배치되어 비행 채취 유도
- BlockTable 가중치(SpawnWeight) 기반 랜덤 배치, 총 40개

3-3. 몬스터 스폰 구조

- SpawnPoint 액터를 에디터에서 배치, ESpawnDir 방향 속성으로 진입 각도 설정
- Combat 진입 시 GameMode가 웨이브 정의(FWaveDef)를 읽어 스폰 펌프 가동
- 스폰 간격마다 랜덤 SpawnPoint에서 몬스터 생성
- 공중 몬스터는 스폰 후 기지 상공 +500uu 고도로 이동 경로 오버라이드

3-4. 업그레이드 시스템 흐름

```

플레이어 구매 요청
→ GameMode::ApplyUpgrade()
  
```

```
→ GameInstance::TryPurchase()    [자원 차감 + 상태 저장]
  → GetShipStats() / GetTurretStats() [파생 스탯 재계산]
    → 다음 웨이브부터 적용
```

- 업그레이드 상태(FUpgradeState)는 GameInstance에 보존 – 재시작에도 유지
- 스탯은 기본값에 업그레이드 보정값을 더하는 방식, 하드코딩 없음

4. UI / HUD 설계

전체 UI는 UMG Canvas 기반 C++ 위젯(NeonHUDWidget)으로 구현. 블루프린트 위젯 없음.

4-1. 플레이 중 HUD

요소	위치	내용
자원 표시	좌상단 (20, 20)	Resources: [수치] 단일 표기
웨이브 레이블	상단 중앙 (0, 20)	Wave N / 5
기지 HP 바	우상단 (-20, 20)	HP 비율에 따라 녹색 → 빨강 전환 (25% 이하 적색)
HP 수치 텍스트	HP 바 아래 우측	Base HP: 현재 / 최대
페이지 오버레이	상단 중앙 (웨이브 아래)	Gather 타이머, PreWave 카운트다운 등
잔여 적 수	상단 중앙 (Combat 중)	Enemies remaining: N
게임오버 / 승리	화면 정중앙	GAME OVER (빨강) / VICTORY! (청록) + [R] Restart

4-2. 터렛 탑승 시 HUD

수동 터렛 탑승 중 이중 크로스헤어 표시:

- 흰색 십자선 (화면 정중앙): 카메라가 향하는 목표 방향
- 노란 사각 점 (포신 투영점): 포신 전방 8000uu 지점을 ProjectWorldLocationToScreen으로 스크린에 투영
- 두 점이 겹칠 때 정확히 명중 – 조준 리드를 직관적으로 시각화

4-3. 상점 HUD

화면 정중앙 720×480 패널, 청록 테두리:

요소	설명
타이틀	UPGRADE SHOP (상단)
자원 표시	타이틀 아래 현재 보유 자원
업그레이드 목록	4열 구성: 이름 / 효과 / 레벨 / 가격
선택 항목	청록색 하이라이트 + > 접두

요소	설명
구매 불가	자원 부족 시 가격 빨간색 표시
최대 레벨	MAX 텍스트, 항목 회색 비활성
조작 힌트	하단 고정: [UP/DOWN] Navigate [ENTER] Buy [TAB] Next Wave

4-4. 메인 메뉴 / 일시정지 / 조작법 패널

패널	크기	항목
메인 메뉴	풀스크린	Start Game / Controls / Quit
일시정지	600×320 중앙	Resume / Controls / Quit Game
조작법	660×480 중앙	전체 키 바인딩 목록

모든 메뉴는 ↑ ↓ 탐색, Enter 선택, ESC 복귀 통일.

5. 기술 구현

5-1. 기술 스택

항목	내용
엔진	Unreal Engine 5
언어	C++ 전용 (블루프린트 게임 로직 없음)
입력	Enhanced Input — C++ 완전 구현 (.uasset 없음)
UI	UMG Canvas Panel — C++ 완전 구현 (블루프린트 위젯 없음)
이펙트	Niagara (선체 트레일), Dynamic Material Instance (MID 색상 제어)
오디오	UAudioComponent (엔진 사운드 속도 연동)

5-2. 클래스 구조

상속 구조

```

AGameModeBase      → ANeonGameMode
UGameInstance      → UNeonGameInstance
APlayerController  → ANeonPlayerController
APawn + IDamageable → APlayerShip
AActor             → AManualTurret
AActor             → AAutoTurret
AActor + IDamageable → ABase (NeonBase)
AActor + IDamageable → AResourceBlock
AActor             → AResourceShard
AActor + IDamageable → AMonster
AHUD               → ANeonHUD
UUserWidget        → UNeonHUDWidget

```

IDamageable 인터페이스

TakeHit(Damage, AttackerPower) 순수 가상 함수. PlayerShip·ManualTurret 사격 시 캐스트 없이 피격 처리 통일.

5-3. 클래스별 함수 목록

ANeonGameMode

게임 전체 흐름을 관장하는 중심 클래스. 상태 머신, 웨이브 스폰, 자원 필드, 업그레이드 적용 담당.

접근	함수 / 변수	설명
public	Phase: EGamePhase	현재 게임 페이지 상태
public	WaveIndex, Resources	현재 웨이브 번호, 보유 자원
public	WaveTable / BlockTable / UpgradeTable	웨이브·블록·업그레이드 인라인 데이터 테이블
public	BeginPlay()	월드 액터 수집, InitDefaultTables, MainMenu 진입
public	Tick()	페이지별 타이머 갱신, Combat 시 SpawnAccum 처리
public	StartGame()	메인 메뉴 → PreWave 전환
public	EnterPhase(NewPhase)	상태 전환 + 진입 시 사이드이펙트 처리
public	RequestStartWave()	Gather 중 F키 입력 시 Combat 조기 진입
public	AddResources(Amount)	자원 증가 (Shard 수집 콜백)
public	OnMonsterKilled()	AliveMonsters 감소, 전원 처리 시 OnWaveCleared 호출
public	OnBaseDestroyed()	기지 HP 0 → GameOver 페이지 전환
public	OnWaveCleared()	마지막 웨이브면 Victory, 아니면 Shop 전환
public	ApplyUpgrade(Id)	TryPurchase 호출 후 전체 액터 스탯 재적용
public	ContinueToNextWave()	Shop → PreWave 전환
private	InitDefaultTables()	WaveTable·BlockTable·UpgradeTable 기본값 코드 초기화
private	SpawnResourceField()	BlockTable 가중치 기반 랜덤 블록 배치
private	TickCombat(Dt)	SpawnAccum 누적, 인터벌 도달 시 SpawnMonsterAtEntrance
private	SpawnMonsterAtEntrance(idx)	SpawnPoint 위치에서 몬스터 생성 및 스탯 주입
private	GetSpawnLocation(Dir)	ESpawnDir → 월드 좌표 변환
private	RefreshAutoTurrets()	업그레이드 후 AutoTurret 스탯 재적용

UNeonGameInstance

업그레이드 상태를 씬 전환 간 영속 보관. 파생 스탯 계산 전담.

접근	함수 / 변수	설명
public	Upgrades: FUpgradeState	TMap<EUpgradeId, int32> 레벨 맵
public	TryPurchase(Id, Resources, Cost, MaxLevel)	자원 차감 + 레벨 증가, 실패 시 false 반환
public	ResetRun()	업그레이드 전체 초기화 (재시작 시)
public	GetShipStats()	기본값 + 업그레이드 보정 → FShipStats 반환
public	GetManualTurretStats()	수동 터렛 파생 스탯 반환
public	GetAutoTurretStats()	자동 터렛 파생 스탯 반환
public	GetAutoTurretCount()	활성화할 AutoTurret 수 반환

APlayerController

입력 액션 생성·등록 및 게임 내 모든 메뉴/상호작용 입력 처리.

접근	함수 / 변수	설명
public	IA_MoveForward/Right/ ThrustUp/Down/Look/Fire/ Ready	비행·사격 입력 액션
public	IA_ShopUp/Down/Confirm/ NextWave/Restart/Interact/ Escape	UI·상호작용 입력 액션
public	BoardedTurret: AManualTurret*	현재 탑승 중인 터렛 참조
public	SetupInputComponent()	IMC 등록, 모든 IA 바인딩
public	Tick()	탑승 중 카메라 ControlRotation → ManualTurret.DesiredAim 전달
public	OnInteract()	근처 ManualTurret 탑승/하차
public	OnShopUp/Down/Confirm()	상점 커서 이동 및 구매
public	OnNextWave()	Tab 입력 → ContinueToNextWave
public	OnEscapePressed()	일시정지 메뉴 토글
private	BuildIMC()	InputMappingContext·InputAction 런타임 생성

APlayerShip

호버크래프트 비행 물리, 사격, 자석 수집 처리.

접근	함수 / 변수	설명
public	Stats: FShipStats	현재 적용 스탯 (업그레이드 반영)
public	ApplyStats(InStats)	업그레이드 후 스탯 갱신
public	TakeHit(Damage, Power)	IDamageable 구현 - HP 감소, 0 이하 시 OnBaseDestroyed
public	OnMoveForward/Right/ThrustUp/Down(Value)	입력값을 float 멤버에 저장
public	OnLook(Value)	마우스 델타 → AddControllerYaw/PitchInput
public	StartFire/StopFire(Value)	bFiring 플래그 제어
public	Tick()	탑승 여부 분기 → 비행 물리 or 터렛 위임
private	FireOnce()	Muzzle → Line Trace → IDamageable.TakeHit
private	CollectShards()	TActorIterator + 거리 ² 비교 → AttractTo 호출

AManualTurret

플레이어 탑승형 수동 터렛. 포신 회전 제한, 카메라 블렌딩 처리.

접근	함수 / 변수	설명
public	DesiredAim: FRotator	매 Tick PlayerController가 기록하는 목표 회전
public	BoardingRadius	탑승 가능 거리 (기본 800uu)
public	SetPlayerBoarded(b)	탑승/하차 - SetViewTargetWithBlend + 카메라 활성화
public	ApplyStats(InStats)	업그레이드 후 터렛 스탯 갱신
public	Fire()	카메라 위치 기준 포신 방향 Line Trace
public	GetBarrelForward()	포신 메시 ForwardVector 반환 (HUD 투영용)
public	Tick()	DesiredAim → 포신 FInterpTo (RotationSpeed 제한)

AAutoTurret

AI 자동 조준·사격 터렛.

접근	함수 / 변수	설명
public	bEnabled	활성 여부 – GameMode가 웨이브별 제어
public	SetEnabled(b)	활성화/비활성화
public	ApplyStats(InStats)	업그레이드 후 스탯 갱신
public	Tick()	FindTarget → FireAtTarget 루프
private	FindTarget()	TActorIterator로 최근접 Monster 탐색
private	FireAtTarget()	Target 방향 Line Trace, FireRate 쿨다운 적용

ABase (NeonBase)

기지 HP 관리 및 시각/청각 피드백.

접근	함수 / 변수	설명
public	MaxHP, CurrentHP	기지 최대·현재 HP
public	TakeHit(Damage, Power)	HP 감소, HitFlashTimer 세트, 0 이하 시 OnBaseDestroyed
public	Tick()	HitFlashTimer 감소 → MID BaseColor 복원, 저 HP 경고색 전환
public	GetHPFraction()	CurrentHP / MaxHP (HUD 바용)

AResourceBlock

공격력 게이트 채굴 블록. 파괴 시 Shard 드롭.

접근	함수 / 변수	설명
public	InitFromDef(Def)	FBlockDef 기반 HP·색상·Shard 수·공격력 요구치 초기화
public	TakeHit(Damage, Power)	AttackerPower < RequiredPower면 SpawnSpark 만, 아니면 HP 감소
public	GetHPFraction()	HUD 월드 스페이스 HP 바용
private	DropShards()	ShardMin~Max 랜덤 수의 ResourceShard 생성
private	SpawnSpark()	공격력 부족 시 시각 피드백

AResourceShard

자석 호밍으로 플레이어에게 흡수되는 자원 파편.

접근	함수 / 변수	설명
public	Value	수집 시 지급 자원량
public	AttractTo(Ship)	HomingTarget 세트 → Tick에서 가속 추적 시작
public	Tick()	HomingTarget 방향 가속 (2000 uu/s ²), 1200uu 이하 진입 시 AddResources 후 Destroy

AMonster

기지 추적 + 원거리 공격 + 지상/공중 변종 이동 패턴.

접근	함수 / 변수	설명
public	HP, MaxHP, MoveSpeed, AttackDPS	스폰 시 WaveDef에서 주입
public	Variant: EMonsterVariant	Ground / Flying — Tick 내 이동 패턴 분기
public	TakeHit(Damage, Power)	HP 감소, 0 이하 시 OnMonsterKilled + Destroy
public	Tick()	기지 방향 이동, AttackRange(2000uu) 진입 시 DPS 공격, Flying은 궤도 이동

5-4. 직접 구현한 시스템 vs 엔진 기능

직접 C++로 구현

시스템	구현 내용
게임 루프 상태 머신	EGamePhase 7단계 (MainMenu → PreWave → Gather → Combat → Shop → Victory/GameOver)
호버크래프트 물리	중력 없는 6자유도 이동, 균일 드래그, z축 자동 감쇠(auto-hover), 속도 기반 선체 틸팅 (Roll/Pitch)
동적 FOV	속도 비율에 따라 70°~110° 선형 보간
엔진 오디오 연동	속도 비율에 따라 피치 0.6x~1.4x, 볼륨 0.3x~1.0x 실시간 변화
Enhanced Input C++	InputAction/MappingContext 완전 코드 생성, .uasset 없이 런타임 등록

시스템	구현 내용
터렛 탑승 시스템	E키 탑승/하차, SetViewTargetWithBlend 카메라 블렌딩, 탑승 중 선체 완전 정지
포신 회전 제한	RotationSpeed 기반 FInterpTo 속도 제한으로 급격한 꺾임 방지
이중 크로스헤어	포신 전방 8000uu 지점을 ProjectWorldLocationToScreen으로 스크린 투영
IDamageable 인터페이스	몬스터·블록·기지·선체 공통 피격 처리, 공격력 게이트 (AttackPower) 채굴 조건
자석 호밍 수집	TActorIterator + 거리 ² 비교로 구현 (Overlap 미사용)
업그레이드 영속화	GameInstance 내 TMap<EUpgradeId, int32> — 씬 전환 후에도 레벨 유지
전체 UI	UMG CanvasPanel + TextBlock/Image를 C++로 직접 생성, 블루프린트 없음
월드 스페이스 HP 바	DrawHUD에서 ProjectWorldLocationToScreen → 몬스터/블록 머리 위 실시간 표시

엔진 기능 활용

기능	활용 방식
Line Trace (ECC_Visibility)	선체/터렛 사격
SetViewTargetWithBlend	터렛 탑승 시 카메라 전환
SpringArm + Camera	선체 3인칭 추적 카메라
Niagara	선체 후방 양쪽 트레일 이펙트
DynamicMaterialInstance	피격 플래시, 기지 경고색 전환
AudioComponent	엔진 사운드 재생

6. 트러블슈팅 / 기술적 도전

3일간의 개발을 시간 순서대로 정리. 각 도전은 초기 접근 → 문제 발견 → 해결 흐름으로 기술.

Day 1 — 엔진 기반 구축

[도전 1] Enhanced Input을 .uasset 없이 C++로 완전 구현

배경: UE5의 Enhanced Input은 에디터에서 .uasset 파일로 IMC/IA를 만드는 것이 일반적인 방식이다. 그러나 에디터 파일 없이 C++ 단독으로 패키징까지 완성하고 싶었다.

시도 1: 처음에는 .uasset을 에디터에서 만들고 ConstructorHelpers::FObjectFinder로 로드하는 방식으로 시작했다. 입력은 동작했지만 에셋 경로 하드코딩과 에디터 파일 의존성이 생겨 유지보수가 불편했다.

시도 2: NewObject<UInputAction>()으로 런타임 생성 시도. 빌드는 됐지만 AddMappingContext 이후 키 이벤트가 전혀 발생하지 않았다. 원인은 IMC 등록 타이밍 — BeginPlay 시점에 EnhancedInputLocalPlayerSubsystem이 아직 준비되지 않은 경우가 있었다.

해결: SetupInputComponent() 오버라이드 안에서 IMC 등록과 IA 바인딩을 함께 처리. BindAction의 핸들러 시그니처를 const FInputActionValue&로 통일해 템플릿 특수화 오류 해결. 결과적으로 .uasset 없이 전체 입력 시스템 완성.

[도전 2] 호버크래프트 물리 — "떠 있는 느낌" 구현

배경: 호버크래프트의 핵심은 중력 없이도 공중에 자연스럽게 떠 있는 느낌이다.

시도 1: UE 기본 MovementComponent(FloatingPawnMovement) 사용. 이동 자체는 됐지만 입력을 놓았을 때 즉시 정지하거나, 반대로 관성이 너무 강해 조종감이 나빴다.

시도 2: SetPhysicsLinearVelocity로 직접 물리 조작. 프레임 간 속도가 불안정해 떨림 현상 발생. 충돌 시 물리 엔진과 간섭으로 예측 불가능한 튕김이 생겼다.

해결: Mesh->SetSimulatePhysics(false), 중력 0, 자체 Velocity 벡터를 Tick에서 수동 적분. 드래그를 $e^{(-drag * dt)}$ 지수 감쇠로 적용해 부드러운 감속 구현. Z축은 입력이 없을 때 FInterpTo로 0에 수렴시켜 자동 고도 유지 효과를 얻었다.

[도전 3] 선체 틸팅과 카메라 독립 회전 충돌

배경: 카메라는 마우스로 자유롭게 회전하면서, 선체는 이동 방향에 따라 자연스럽게 기울어져야 했다.

문제: `bUseControllerRotationYaw = true`로 설정하면 선체가 카메라 Yaw를 그대로 따라가 별도 틸팅 연출이 불가능했다. 반대로 `false`로 두면 선체가 항상 월드 X축을 바라보며 고정됐다.

해결: `bUseControllerRotationYaw = false`로 두고, Tick에서 `ControlRotation.Yaw`만 선택적으로 선체 회전에 반영. Roll은 `MoveRightInput × 18°`, Pitch는 `ControlRotation.Pitch`를 $\pm 25^\circ$ 클램프해 `FRInterpTo`로 보간. 카메라는 `SpringArm`이 `ControlRotation`을 다르게 해 독립 유지.

Day 2 — 핵심 메카닉 구현

[도전 4] 수동 터렛 탑승 – 카메라 전환과 입력 라우팅

배경: E키로 터렛에 탑승하면 카메라가 터렛 후방으로 이동하고, LMB는 선체 사격이 아닌 터렛 사격이 되어야 했다.

시도 1: 탑승 시 선체 Camera를 비활성화하고 터렛 Camera를 활성화하는 방식. 카메라 전환이 즉각적으로 일어나 툭 끊기는 느낌이 있었다.

시도 2: `SetViewTarget`으로 뷰 타겟을 터렛 액터로 변경. 블렌딩이 없어 동일하게 끊겼다.

해결: `SetViewTargetWithBlend(Turret, 0.25f)`로 0.25초 블렌딩 전환. 탑승 중 `PlayerShip::Tick`에서 `BoardedTurret`이 있으면 `Velocity = Zero` 및 조기 `return`으로 선체 완전 정지. `PlayerController::Tick`에서 카메라 `ControlRotation`을 `ManualTurret.DesiredAim`에 매 프레임 기록해 포신이 마우스를 따라오도록 구현.

[도전 5] 이중 크로스헤어 – 포신 방향 스크린 투영

배경: 수동 터렛 탑승 중 "카메라가 보는 방향"과 "포신이 실제로 향하는 방향"이 다를 때 이를 동시에 표시해야 했다.

시도 1: `DrawDebugSphere`를 포신 끝 월드 좌표에 그리는 방식. 3D 공간에 구체가 그려져 앞의 물체에 가려지거나, 시야각에 따라 크기가 변해 혼란스러웠다.

해결: `PlayerController->ProjectWorldLocationToScreen(BarrelPos, ScreenPos)`로 포신 전방 8000uu 지점을 2D 스크린 좌표로 변환. `DrawHUD`에서 해당 위치에 5px 노란 사각형을 그려 항상 화면 위에 일관된 크기로 표시. 화면 중앙 흰 십자선(카메라 방향)과 함께 두 크로스헤어가 겹치면 명중한다는 직관적 UX 완성.

[도전 6] 자원 수집 – SphereOverlap 대신 TActorIterator 선택

배경: 자석 반경 내 ResourceShard를 탐지하는 방법을 결정해야 했다.

시도 1: GetWorld()->OverlapMultiByChannel로 구형 범위 내 오버랩 탐지 시도. FOverlapResult 구조체가 WorldCollision.h에 전방 선언만 되어 있고 실 정의가 다른 헤더에 있어 컴파일 오류 발생. 필요한 include 체인이 복잡했다.

해결: TActorIterator<AResourceShard>로 월드 내 모든 Shard를 순회하며 FVector::DistSquared로 거리 비교. Shard 수가 많지 않아 성능 문제 없음. 코드가 단순해지고 헤더 의존성도 사라짐.

[도전 7] 게임 루프 상태 관리 – 플래그 지옥 탈출

배경: 초기에는 bInCombat, bInShop, bWaveCleared 등의 bool 플래그를 여러 클래스에 분산해 관리했다.

문제: 플래그 조합이 늘어날수록 Shop 중에 몬스터가 스폰되는 등 예상치 못한 상태 충돌이 발생했다. 어느 클래스가 진실의 소스인지도 불분명해졌다.

해결: EGamePhase 열거형 하나로 가능한 상태를 모두 정의하고, GameMode가 유일한 소스가 되도록 리팩터. EnterPhase(NewPhase) 함수 한 곳에서 전환 시 사이드이펙트를 처리해 다른 클래스는 GM->Phase만 읽으면 되게 정리. 이후 상태 충돌 버그가 사라졌다.

[도전 8] 공중 몬스터 이동 패턴 – 직선 이동의 단조로움 해결

배경: 공중 몬스터(Flying Variant)를 지상과 동일하게 직선 이동시켰더니 시각적으로 지상 몬스터와 구분이 안 됐다.

시도 1: 무작위 방향으로 튀기는 Wander 패턴 적용. 기지에 도달하지 못하고 맵 밖으로 벗어나는 경우 발생.

해결: 기지를 중심으로 반경을 좁혀가는 나선 궤도(OrbitAngle, OrbitRadius) 이동 구현. WanderPhase 사인 함수로 미세한 상하 흔들림 추가. 기지로 수렴하면서도 예측 불가능한 움직임을 연출해 지상 몬스터와 명확히 구별됨.

Day 3 — UI 및 마무리

[도전 9] UMG 위젯 전체를 C++로 – 블루프린트 없는 UI 구현

배경: 블루프린트 없이 C++만으로 폴스크린 메뉴, 상점, HUD를 모두 구현하는 것이 목표였다.

시도 1: UUserWidget을 상속한 클래스를 AddToViewport로 추가하고 Canvas에 위젯을 동적 생성. NativeOnInitialized가 예상보다 일찍 호출돼 WidgetTree가 아직 유효하지 않은 타이밍 오류 발생.

해결: BuildLayout()을 NativeOnInitialized() 안에서만 호출되도록 정리. PlaceAt() 헬퍼 함수로 CanvasPanelSlot 앵커·정렬·위치·크기를 한 줄에 설정해 반복 코드 최소화. NativeTick에서 GameMode->Phase 기반으로 패널 Visibility를 매 프레임 갱신하는 방식으로 상태별 UI 전환 완성.

[도전 10] Enter·방향키가 게임 입력과 충돌

배경: 상점에서 UMG 위젯이 AddToViewport되자 Enter·방향키 입력이 위젯 포커스 시스템으로 먼저 소비돼 게임 입력 핸들러에 도달하지 않았다.

해결: PC->SetInputMode(FInputModeGameOnly()) 호출로 위젯이 키보드 포커스를 가져가지 않도록 강제. 상점 탐색은 Enhanced Input으로 직접 처리해 UMG 기본 포커스 체계와 완전히 분리.